TradXSell- Static Analysis Report

by

Kashaf Fatima 22i-2415

Amna Noor 22i-1529

Shiza Najeeb 22i-1532

Software Quality Engineering

Semester Project

Submitted to [Sir Atif Jilani]

Date: 12/11/2024

Static Analysis:

Routes:
CartRoutes.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	No	CartSchema.js is not written in PascalCase.	Rename the file to CartSchema.js following PascalCase.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	None	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.
4	House Keeping	Are there any code smells not covered by the checklist that are present in the code?	Yes	Repeated use of CartItem.findOne in multiple methods.	Extract the CartItem.findOne logic into a helper method to reduce duplication.
5	House Keeping	Are there any violations of coding standards not covered by the checklist?	No	None	No changes needed.

6	House Keeping	Are there any performance inefficiencies not covered by the checklist?	Yes	Use of await within loops can be replaced with Promise.all for concurrent processing where possible.	Refactor to use Promise.all for operations like fetching or saving multiple cart items concurrently.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Repeated calls to CartItem.findOne in multiple methods.	Extract the logic into a reusable helper method to reduce redundancy.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	The use of await for individual database operations prevents concurrent execution.	Refactor to use Promise.all for concurrent processing where feasible, especially for batch operations.

Complainroute.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix

1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	No	complainschema.js is not written in PascalCase.	Rename the file to ComplainSchema.js to follow PascalCase naming conventions.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	None	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	No input validation is performed for data length or structure, leading to potential issues.	Add validation logic to ensure complaints have a minimum length, and emails match a valid format.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	None	No changes needed.

OrderRoutes.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	None	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	None	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Repeated logic for updating order statuses (status: 'Shipped' and status: 'Delivered') in similar routes.	Extract the common logic for updating statuses into a helper function for reusability and maintainability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	The use of findByIdAndUpdate for updating all item statuses may not scale well for large datasets.	Consider optimizing the update query logic or batching updates if scalability becomes a concern.

ProductRoutes.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	None	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	None	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Repeated queries for fetching similar product data by different attributes (e.g., category, email).	Refactor to create a reusable function for querying products by different attributes for better maintainability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Inefficient category comparisons using .toLowerCase() and direct string comparisons in multiple routes.	Use indexed fields or a case-insensitive query in the database for category filtering to improve performance.

ReviewRoutes.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	None	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	None	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	The productId is repeatedly extracted and used without validation.	Add validation to ensure productId is not null or malformed before querying the database.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	None	No changes needed.

SellerRoutes.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	None	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	None	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Repeated logic for fetching productIds from Product.find() is present in multiple routes.	Extract a reusable helper function for fetching productIds by sellerEmail to reduce code duplication.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Fetching all products and mapping productIds before queries may cause inefficiencies with large datasets.	Use MongoDB aggregation or indexed queries to fetch and process data more efficiently.

UserRoutes.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	None	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	None	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Passwords are stored in plain text, which is a major security risk.	Implement password hashing using libraries like bcrypt to ensure secure password storage.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	The use of findOne in multiple routes may cause unnecessary database queries if not optimized.	Consider adding indexes to the email and _id fields in the database to optimize queries.

Schemas:

CartSchema.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	None	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	None	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	No	None	No changes needed.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Indexes are not defined for frequently queried fields like email and productId.	Add indexes to email and productId fields for faster query performance in MongoDB.

complainschema.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix

1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	None	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	None	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	No	None	No changes needed.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Indexes are not defined for the email field, which is likely queried frequently.	Add an index to the email field to improve query performance in MongoDB.

OrderSchema.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	None	No changes needed.

2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	None	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	The status field for individual items and the entire order may lead to inconsistency.	Implement logic or triggers to ensure the status of the order reflects the aggregated item statuses.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Indexes are not defined for commonly queried fields like email, items.productId, and orderDate.	Add indexes to email, items.productId, and orderDate for improved query performance in MongoDB.

ProductSchema.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix

1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	None	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	None	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	The status field for products has multiple states, which may lead to confusion or complexity in handling states.	Consider simplifying the status field or adding more structured validation to prevent invalid states.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Indexes are not defined for commonly queried fields such as id, sellerEmail, and category.	Add indexes to the id, sellerEmail, and category fields for faster query performance in MongoDB.

ReviewSchema.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	None	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	None	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	No	None	No changes needed.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	The productId, userEmail, and username fields may be queried frequently without proper indexing.	Add indexes to productId, userEmail, and username fields to improve query performance in MongoDB.

UserSchema.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	None	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	None	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Password is stored in plain text, which is a security concern.	Use a hashing algorithm like bcrypt to securely hash the password before storing it in the database.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Indexes are not defined for frequently queried fields like email and username.	Add indexes to email and username fields to improve query performance in MongoDB.

Server.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	None	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	None	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	No error handling or validation for the MongoDB connection.	Add error handling for cases where the MongoDB connection fails or there are issues with the network.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	CORS is configured to allow access only from localhost:3000. This may need to be more dynamic for production environments.	Configure CORS with environment variables to dynamically allow access based on different environments.

Components:

Admin:

complain.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name Complains follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable names and method names (e.g., handleSubmit, setComplaint) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	There are no constants in the file.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	The handleSubmit function contains the logic for making API requests and handling UI updates, which can be separated for better readability and maintainability.	Refactor the code by separating the logic into smaller functions (e.g., an API call handler and a UI update function).
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.

Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	None	No changes needed.
Error Handling	Is error handling implemented effectively?	Yes	The code has proper error handling for the fetch request, ensuring users get feedback on submission errors.	No changes needed.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled effectively and consistently with user experience in mind.	No changes needed.

ManageOrders.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name ManageOrders follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable names and method names (e.g., fetchOrders, setOrders, handleItemStatusUpdate) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	fetchProductSellerEmails handles multiple asynchronous operations that could be split into smaller functions.	Refactor fetchProductSellerEmails to separate concerns (API calls and state updates).
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	The useEffect hook fetches orders on initial load and triggers multiple subsequent API calls for each product.	Optimize API calls by batching requests or using a more efficient data fetching strategy.
Error Handling	Is error handling implemented effectively?	Yes	Error handling is implemented for both order fetching and product seller email fetching.	No changes needed.

UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled well, but large useEffect hooks or long forms can affect the user experience.	Split loading logic to improve responsiveness and avoid blocking UI.

ManageProducts.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The class name ManageProducts follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable names and method names (e.g., handleChange, handleSubmit, fetchProducts) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	The handleSubmit function contains business logic and form handling, which could be split into separate functions for better readability and maintainability.	Refactor the handleSubmit function to separate form validation, API calls, and state updates.

Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	The componentDidMount method triggers an API call on mount, but there are no loading indicators for the user when the products are being fetched.	Add a loading state to indicate to the user that the products are being fetched.
Error Handling	Is error handling implemented effectively?	Yes	The component handles errors by updating the state with an error message in case of failed API calls.	No changes needed.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The UI components are styled well but could be improved by ensuring form validation feedback is provided to the user when adding a product.	Implement form validation feedback to inform the user about missing or incorrect fields when submitting the form.

ProductDetails.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name ProductDetails follows PascalCase.	No changes needed.

2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable names and method names (e.g., fetchReviews, setProduct, handleSubmit) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	The useEffect hook is used to fetch product details and reviews, making it a bit complex and could be separated into individual functions for better maintainability.	Refactor to separate concerns (e.g., creating a custom hook for fetching reviews and product details).
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Fetching product and review data separately in useEffect could be optimized by combining them into a single API call.	Combine product and review fetching into a single API call to reduce redundant requests.
Error Handling	Is error handling implemented effectively?	Yes	Error handling is implemented for both product fetching and review fetching.	No changes needed.

UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is well-styled, but the loadingMessage and errorMessage could be made more prominent for better UX.	Increase visibility of the loading/error messages by improving styling or adding animations.

Register.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name Register follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable names and method names (e.g., handleRegisterValidation, setUsername) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles and setTimeout used for handling success message and navigation, which can be refactored.	Extract styles into a CSS/SCSS file, and consider using useEffect with a timeout state for better management.

Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Repeated state updates for handling input values could be managed better.	Use a single useState with an object to handle multiple input fields efficiently.
Error Handling	Is error handling implemented effectively?	Yes	Error handling for registration is present and provides user-friendly feedback.	No changes needed.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled effectively but could benefit from improved validation error messaging.	Add specific error messages for invalid email or password formats.

SellerDashboard.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name SellerDashboard follows PascalCase.	No changes needed.

2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable names and method names (e.g., fetchData, setDailyOrders) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Multiple fetch calls for seller data could be batched or consolidated into a single API request.	Create a single endpoint to fetch all necessary seller data in one call to improve efficiency.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Repeated calculations of the maximum daily orders for the y-axis in the lineOptions.	Compute the maximum value for dailyOrders once before defining the chart options.
Error Handling	Is error handling implemented effectively?	Yes	Error handling is implemented for all fetch requests.	No changes needed.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The dashboard is well-styled, but accessibility could be enhanced with tooltips or screen reader support.	Add ARIA labels and tooltips for better accessibility.

Sidebar.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name SideNavbar follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., handleLogout, navLink) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are heavily used, which can make the component less maintainable.	Extract styles into a separate CSS or SCSS file for better maintainability and reusability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.

Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	None	No changes needed.
Error Handling	Is error handling implemented effectively?	Not Applicable	There is no error-prone logic in this component.	No changes needed.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled effectively, but it could benefit from hover effects for the logout button and navigation links.	Add hover effects to enhance interactivity and user experience.

SideNavbar.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name SideNavbar follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., handleLogout, navLink) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are heavily used, which can make the component less maintainable.	Extract styles into a separate CSS or SCSS file for better maintainability and reusability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	None	No changes needed.
Error Handling	Is error handling implemented effectively?	Not Applicable	There is no error-prone logic in this component.	No changes needed.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled effectively, but it could benefit from hover effects for the logout button and navigation links.	Add hover effects to enhance interactivity and user experience.

MainAdmin:
Addquality.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix

1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name Register follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., handleRegisterValidation, setSuccessMessage) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used extensively, reducing maintainability.	Extract styles into a separate CSS or SCSS file for better readability and reusability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	None	No changes needed.

Error Handling	Is error handling implemented effectively?	Yes	Error messages are displayed effectively, but specific field validation messages are missing.	Add field-specific validation messages for improved user experience.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled effectively but could benefit from hover effects for buttons.	Add hover effects to improve interactivity and visual feedback for users.

AdminHeader.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name AdminHeader follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., userProfile, iconButton) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix

Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used extensively, making the component harder to maintain.	Extract styles into a separate CSS/SCSS file for better maintainability and scalability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	None	No changes needed.
Error Handling	Is error handling implemented effectively?	Not Applicable	No error-prone logic in this component.	No changes needed.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled effectively but could use hover effects for buttons.	Add hover effects to improve user interaction feedback.

CheckComplaints.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name Checkcomplaints follows PascalCase.	No changes

					needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., handleSidebarToggle, fetchComplaints) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used extensively, which can make the code less maintainable.	Extract inline styles into a separate CSS/SCSS file for better maintainability and scalability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	None	No changes needed.
Error Handling	Is error handling implemented effectively?	Yes	Error handling is implemented for the axios.get function, but no user feedback is provided for errors.	Display an error message or notification to users when complaints fail to load.

UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled effectively, but it could benefit from responsive design improvements.	Add responsive design to enhance usability on smaller screens.

CheckProduct.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name ManageProducts follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., handleStatusChange, fetchProducts) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

Housekeeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used extensively, which can make the code less maintainable.	Extract inline styles into a separate CSS/SCSS file for better maintainability and scalability.

Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	None	No changes needed.
Error Handling	Is error handling implemented effectively?	Yes	Error handling is implemented for API calls but lacks user feedback in some cases.	Display error notifications or messages to users when an action fails.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled effectively but could benefit from responsive design improvements.	Add responsive design to enhance usability on smaller screens.

CheckSellers.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name CheckSellers follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., handleDetailsClick, fetchSellers) follow camelCase.	No changes needed.

3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used extensively, which can make the code less maintainable.	Extract inline styles into a separate CSS/SCSS file for better maintainability and scalability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present?	No	None	No changes needed.
Error Handling	Is error handling implemented effectively?	Yes	Error handling is implemented for the axios.get and axios.delete functions, but user feedback for errors is missing.	Display an error message or notification to users when sellers fail to load or delete operations fail.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled effectively, but could benefit from responsive design improvements.	Add responsive design to enhance usability on smaller screens.

QualityWale:

Complains.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name Complains follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., fetchComplaints, setComplaints) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

Housekeeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used extensively, which can make the code less maintainable.	Extract inline styles into a separate CSS/SCSS file for better maintainability and scalability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	None	No changes needed.

Error Handling	Is error handling implemented effectively?	Yes	Error handling is implemented for the axios.get function, but no user feedback is provided for errors.	Display an error message or notification to users when complaints fail to load.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled effectively, but it uses static widths and colors, reducing flexibility.	Add responsive design improvements for better usability on different screen sizes and themes.

ManageProductsQuality.js:

JavaScript-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name ManageProducts follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., fetchProducts, setProducts, handleStatusChange) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are defined in the provided code.	No changes needed.
4	React Best Practices	Are hooks (useState, useEffect) being used correctly?	Yes	Hooks are used correctly for state management and side effects.	No changes needed.

5	React Best Practices	Are props used efficiently (if applicable)?	Yes	The component correctly uses props for rendering and event handling in the ProductDetailsPopup.	No changes needed.

Housekeeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used extensively, making the code harder to maintain.	Extract inline styles into a separate CSS/SCSS file for better readability and maintainability.
Error Handling	Is error handling implemented effectively?	Partial	Error handling exists for axios calls, but no feedback is given to users for errors.	Implement user notifications or error messages for better user experience.
Code Reusability	Are reusable components/functions being used?	Partial	Repetitive style objects (tableHeaderStyle, tableCellStyle, etc.) could be extracted into a shared module.	Define these shared styles/constants in a separate file and import them to improve reusability.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Filters (typeFilter, statusFilter) are recalculated on every render, impacting performance.	Memoize filteredProducts using useMemo for optimization.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	Components are styled effectively but lack responsiveness for smaller screens.	Add responsive design improvements (e.g., CSS media queries or a library like Bootstrap).

SellerDetailsPopup.js:

JavaScript-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name SellerDetailsPopup follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variables (popupStyle, popupContentStyle, etc.) and method names (onClose) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are defined in the provided code.	No changes needed.
4	React Best Practices	Are props destructured for easier readability and usage?	Yes	Props (seller, onClose) are destructured in the function parameter.	No changes needed.
5	React Best Practices	Are inline functions used appropriately?	Yes	The onClick handler for the close button is properly defined.	No changes needed.
6	React Best Practices	Are props validated (e.g., using PropTypes)?	No	Props are not validated using PropTypes or TypeScript.	Add PropTypes for prop validation to improve type safety.

Housekeeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used, making the code harder to maintain.	Extract inline styles into a CSS/SCSS file for better

				readability and maintainability.
Error Handling	Is error handling implemented effectively?	Not Applicable	No error handling is needed in this component.	No changes needed.
Code Reusability	Are reusable components/functions being used?	No	Styles are repeated directly in the component.	Extract reusable styles to a shared CSS module or constants file.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	The component is simple and does not have significant performance issues.	No changes needed.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The popup is visually appealing but not responsive for smaller screens.	Add responsive design improvements, e.g., CSS media queries for smaller devices.

Aboutus.js:

JavaScript-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., AboutUs)?	Yes	The component name Aboutus follows PascalCase but should ideally be AboutUs.	Rename Aboutus to AboutUs for better naming consistency.

2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Not Applicable	No variable or method names in this component.	No changes needed.
3	React Best Practices	Is JSX code organized and readable?	Yes	JSX code is readable but uses inline styles extensively.	Extract styles to a CSS/SCSS file for better readability and maintainability.
4	React Best Practices	Are inline functions used appropriately?	Yes	No inline functions are used, which is appropriate for this component.	No changes needed.
5	React Best Practices	Are semantic HTML elements used correctly?	Yes	Semantic HTML elements like h1, h2, and ul are used correctly.	No changes needed.
6	React Best Practices	Are props validated (e.g., using PropTypes)?	Not Applicable	This component does not accept any props.	No changes needed.

Housekeeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Extensive use of inline styles makes the code less maintainable.	Extract styles into a CSS/SCSS file for easier maintenance and readability.
Error Handling	Is error handling implemented effectively?	Not Applicable	No error handling is required in this static component.	No changes needed.

Code Reusability	Are reusable components/functions being used?	No	No reusable components are utilized for repeated styling or structure.	Create reusable components (e.g., for section with h2 and p structure) to reduce redundancy.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	This is a static component with no performance concerns.	No changes needed.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Partially	The design is consistent but could benefit from responsive enhancements.	Add responsive styles (e.g., CSS media queries) for better viewing on various screen sizes.

Cart.js:

JavaScript-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are variable names descriptive and written in camelCase (e.g., calculateTotalPrice)?	Yes	Variables like increaseQuantity and cartItems follow camelCase.	No changes needed.
2	Readability	Is the code formatted consistently and easy to read?	Yes	Code is well-formatted and readable.	No changes needed.
3	Dependency Management	Are dependencies used correctly and efficiently?	Yes	All dependencies are relevant.	No changes needed.

4	React Best Practices	Are useEffect dependencies set correctly?	Yes	The useEffect hook correctly includes email and navigate in the dependency array.	No changes needed.
5	React Best Practices	Are props and context used effectively?	Yes	The AuthContext is used effectively to get email.	No changes needed.
6	React Best Practices	Is state management handled effectively with useState?	Yes	useState is used appropriately for cartItems and error.	No changes needed.
7	API Integration	Are API calls handled effectively (e.g., error handling, async/await)?	Yes	API calls include error handling and async/await is used appropriately.	No changes needed.

Housekeeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist?	Yes	Extensive use of inline styles makes the code less maintainable.	Extract styles into a CSS/SCSS file for easier maintenance and readability.
Error Handling	Is error handling implemented effectively?	Yes	Error handling is implemented for API calls.	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Mapping through cartItems multiple times (e.g., for reduce and rendering).	Memoize calculations like the total price using useMemo for better performance.
Code Reusability	Are reusable components/functions being used?	No	Reusable components are not used for repeated elements like rows or buttons.	Create reusable components for repeated structures like cart item rows or buttons to improve maintainability.

UI/UX Considerations	Are UI components styled effectively and with consistency?	Partially	Design is consistent but relies heavily on inline styles, limiting reusability.	Extract styles into CSS/SCSS and add responsive designs for smaller screens.

Category.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name CategoryCarousel follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., handlePrev, handleNext) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used extensively, which can make the code less maintainable.	Extract inline styles into a separate CSS/SCSS file for better maintainability and scalability.

Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	None	No changes needed.
Error Handling	Is error handling implemented effectively?	No	No error handling is implemented for user interactions (like page navigation).	Consider adding user feedback for errors, such as disabling buttons when no pages are available.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled effectively, but it could benefit from responsiveness on smaller screens.	Add responsive design to ensure the layout is user-friendly across devices.

CategoryPage.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		The component name CategoryPage follows PascalCase. No changes needed.

2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		Variable and method names (e.g., fetchProducts, setLoading) follow camelCase. No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable		No constants are used in the provided code. No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used extensively.	Extract inline styles into a separate CSS/SCSS file for better maintainability and scalability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No		No violations found.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No		No performance inefficiencies found.
Error Handling	Is error handling implemented effectively?	Yes		Error handling is implemented for the fetch request, but no user feedback is provided for errors. Display an error message or notification

				to users when products fail to load.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes		The component is styled effectively, but responsiveness can be improved for smaller screen sizes. Add responsive design improvements.

Checkout.js:

JavaScript-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		The component name Checkout follows PascalCase. No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		Variable and method names (e.g., fetchCartItems, handleOrder) follow camelCase. No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable		No constants are used in the provided code. No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are	Yes	The inline styles and hardcoded API endpoints may lead to	Extract inline styles into a CSS/SCSS file, and parameterize API endpoints for reusability.

	present in the code?		maintainability issues.	
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No		No violations found.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No		No performance inefficiencies found.
Error Handling	Is error handling implemented effectively?	Yes		Error handling is implemented for fetching cart items and placing orders. Ensure user-friendly feedback (e.g., modal, notification).
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes		The UI components are styled effectively, but adding CSS classes for reusability and responsiveness would be a good improvement.

Details.js:

JavaScript-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		The component name Details follows PascalCase. No changes needed.

2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		Variable and method names (e.g., fetchProduct, addToCart, handleQuantityChange) follow camelCase. No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable		No constants are used in the provided code. No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used throughout the components. Repetitive styles for select dropdowns, inputs, and buttons.	Move the inline styles to a CSS/SCSS file for better maintainability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No		No violations found.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No		No performance inefficiencies found.
Error Handling	Is error handling implemented effectively?	Yes		Error handling is implemented for fetching product details. Consider adding more detailed

				user feedback, such as a retry option.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes		The UI components are effectively styled, but a CSS/SCSS file would help improve code consistency and reusability for large projects.

Footer.js:

JavaScript-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		The component name Footer follows PascalCase. No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		The variable names (e.g., text-center, background-color) follow camelCase for inline styles and HTML attributes. No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable		No constants are used in the provided code. No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are	Yes	Inline styles used repetitively, which can affect maintainability for	Move the inline styles to a CSS/SCSS file for better maintainability and cleaner code.

	present in the code?		large-scale projects.	
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No		No violations found.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No		No performance inefficiencies found.
Error Handling	Is error handling implemented effectively?	N/A		Not applicable as no asynchronous logic or external calls are used in this component.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes		The UI components are consistently styled with a focus on layout and design, but inline styles could be moved to external CSS for maintainability.

ForgotPassword.js:

JavaScript-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		The component name ForgotPassword follows PascalCase. No changes needed.

2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		The variable names (e.g., email, newPassword, message) follow camelCase. No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable		No constants are used in the provided code. No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles used repetitively.	Move the inline styles to a CSS/SCSS file for better maintainability and readability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No		No violations found.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No		No performance inefficiencies found.
Error Handling	Is error handling implemented effectively?	Yes		The try...catch block for handling errors during password update is correctly implemented.

UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes		The UI components are consistently styled with a focus on layout, but inline styles could be moved to external CSS for maintainability.

Home.js:

JavaScript-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		The component names like CallToAction, Home, Category, and Footer follow PascalCase. No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		Variable and method names like products, setProducts, fetchProducts, etc., follow camelCase. No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	No		The counts object could be written in uppercase as a constant for consistency.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles used repetitively.	Move the inline styles to a CSS/SCSS file for better maintainability and readability.

Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No		No violations found.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No		No performance inefficiencies found.
Error Handling	Is error handling implemented effectively?	Yes		The try...catch block for handling errors during product fetching is correctly implemented.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes		Inline styles could be replaced with external CSS, but the component layout is consistent and clear.
Accessibility	Are there any accessibility issues in the component?	No		The code does not have major accessibility issues, but using semantic HTML tags and ARIA attributes could improve it further.
Responsiveness	Is the UI responsive?	Yes		The layout uses flexible units (like flex and grid) that work well for responsiveness, but CSS refactor could enhance scalability.

Login.js:

JavaScript-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		The component names like AuthPage, Loginn, Register, and ForgotPassword follow PascalCase. No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		The variable names like isRegister, setIsRegister, isForgotPassword, setIsForgotPassword follow camelCase. No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	No		No constants are used here, but if added, they should follow uppercase with underscores.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	No		No code smells found.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No		No violations found.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No		No performance inefficiencies found.
Error Handling	Is error handling implemented effectively?	No		Error handling is not required for this component as it just

				manages which form to display.
UI/UX Considerations	Are UI components styled effectively and with consistency?	No		No UI styles are provided here. Consider adding styles for better UX, or pass styles from parent components.
Accessibility	Are there any accessibility issues in the component?	No		The component itself does not have accessibility issues, but it could benefit from adding semantic HTML elements (e.g., main, section).
Responsiveness	Is the UI responsive?	No		This component is likely a part of a larger page, so responsiveness depends on the parent layout. No specific responsiveness issues.

LoginPage.js:

JavaScript-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		The component names like Login follow PascalCase. No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		The variable names like email, password, showPassword, handleLoginValidation, showAlert, and handleLogin follow camelCase.

3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	No		No constants are used here, but if added, they should follow uppercase with underscores.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	No		No code smells found.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No		No violations found.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No		No performance inefficiencies found.
Error Handling	Is error handling implemented effectively?	Yes		Error handling is implemented to show an alert if login fails. No changes needed.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes		The styles provided are effective, though they are in-line. Consider moving them to external CSS files for maintainability.
Accessibility	Are there any accessibility issues in the component?	No		The component is functional with no major accessibility issues, but could benefit from aria-labels on interactive elements for clarity.

Responsiveness	Is the UI responsive?	Yes		The component layout uses flexbox, making it responsive. No specific responsiveness issues.

Mens_products.js:

JavaScript-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		The component Mens_products follows PascalCase naming convention.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		The variable names like products, searchQuery, error, fetchProducts, and handleSearch follow camelCase.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	No		No constants are used here, but if added, they should follow uppercase with underscores.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	No		No code smells found.
Coding Standards	Are there any violations of coding standards not covered by the	No		No violations found.

	checklist that are present in the code?			
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No		No performance inefficiencies found.
Error Handling	Is error handling implemented effectively?	Yes		Error handling is implemented via try-catch block, and the error message is displayed if the fetch operation fails. No changes needed.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes		The component is styled with inline styles, but it would benefit from moving to CSS classes for better maintainability.
Accessibility	Are there any accessibility issues in the component?	No		No major accessibility issues. Could consider adding alt text to the product images for better accessibility.
Responsiveness	Is the UI responsive?	Yes		The layout uses flexible container widths (80%), but it would be more responsive if external CSS handled different screen sizes.

Navigationbar.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name Navigationbar	No changes needed.

				follows PascalCase.	
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., handleLogout) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used extensively, which can make the code less maintainable.	Extract inline styles into a separate CSS/SCSS file for better maintainability and scalability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	None	No changes needed.
Error Handling	Is error handling implemented effectively?	Not Applicable	No error handling required in this file.	No changes needed.

UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled effectively, but it could benefit from responsive design improvements.	Add responsive design to enhance usability on smaller screens.

NewArrivals.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name NewArrivals follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., fetchProducts, handleSearch) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used extensively, which can make the code less maintainable.	Extract inline styles into a separate CSS/SCSS file for better maintainability and scalability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are	No	None	No changes needed.

	present in the code?			
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	None	No changes needed.
Error Handling	Is error handling implemented effectively?	Yes	Error handling is implemented for the axios.get function, but there is no user feedback for failed requests.	Display an error message or notification to users when products fail to load.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled effectively, but it could benefit from responsive design improvements.	Add responsive design to enhance usability on smaller screens.

Order.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name Order follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., N/A) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used extensively, which can make the code less maintainable.	Extract inline styles into a separate CSS/SCSS file for better maintainability and scalability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	None	No changes needed.
Error Handling	Is error handling implemented effectively?	No	No error handling is implemented as there are no dynamic data fetching operations.	No changes needed.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled simply but effectively.	Consider adding more content or context for users, such as a button to return to the homepage or view order details.

ProductsDetailspopup.js

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix

1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name ProductDetailsPopup follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., onClose, popupOverlayStyle) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Yes	Constant variables (e.g., popupOverlayStyle, popupStyle, closeButtonStyle) are written in camelCase, which is acceptable for local styling constants.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used for the overlay, popup, and button styles. This can affect code maintainability and reusability.	Extract inline styles into a separate CSS/SCSS file for better maintainability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are	No	None	No changes needed.

	present in the code?			
Error Handling	Is error handling implemented effectively?	Yes	The component checks if the product object is not present and returns null, preventing rendering errors.	No changes needed.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component uses a simple and consistent style, with clear buttons and a responsive layout.	Consider adding animations or transitions when opening/closing the popup to enhance user experience.

RegisterPage.js:

JavaScript-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are component names written in PascalCase (e.g., RegisterForm)?	Yes	Component name Register follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., getUser)?	Yes	Variable and method names (e.g., handleRegisterValidation) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., API_URL)?	Not Applicable	No constants are used in the provided code.	No changes needed.

Housekeeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix

Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used extensively.	Extract inline styles into a separate CSS/SCSS file for better maintainability and scalability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None.	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Animation keyframes are defined within the component.	Move keyframes to an external CSS/SCSS file for better performance and reuse.
Error Handling	Is error handling implemented effectively?	Yes	Error handling is implemented but could include more user-friendly notifications.	Use toast notifications or modal dialogs to show errors/success messages instead of inline alerts.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled effectively but lacks responsiveness for smaller screens.	Add responsive styles using media queries or CSS frameworks like TailwindCSS or Bootstrap.

Review.js:

JavaScript-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are component names written in PascalCase (e.g., OrderService)?	Yes	The component name Reviews	No change

				follows PascalCase.	s needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., fetchReviews, handleSubmit) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

Housekeeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present?	Yes	Inline styles are used extensively, which can make the code less maintainable.	Extract inline styles into a separate CSS/SCSS file for better maintainability.
Coding Standards	Are there any violations of coding standards not covered by the checklist?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist?	No	None	No changes needed.
Error Handling	Is error handling implemented effectively?	Yes	Errors are logged, but no user feedback is provided for fetch failure.	Display an error message or notification to users when reviews fail to load.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	UI is styled effectively, but could benefit from separating styles	Extract styles to a separate file and add responsive design for better user experience.

			and adding responsiveness.	

UserComplaints.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name Complains follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., handleSubmit, setComplaint) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

Housekeeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used extensively, which can make the code less maintainable.	Extract inline styles into a separate CSS/SCSS file for better maintainability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present?	No	None	No changes needed.

Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist?	No	None	No changes needed.
Error Handling	Is error handling implemented effectively?	Yes	Error handling is implemented, but user feedback can be improved visually.	Add styles to display user messages more prominently.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled effectively but lacks responsive design improvements.	Add responsive design styles for better usability on smaller screens.

UserDashboard.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name UserDashboard follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., handleEditClick, saveChanges, fetchInfo) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	No	Constants like buttonStyle and editButtonStyle are not uppercase.	Refactor constant definitions to uppercase with underscores, e.g.,

					BUTTON_STYLE.

Housekeeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used extensively, making the code less maintainable.	Extract inline styles into a separate CSS/SCSS file for better maintainability and scalability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	None	No changes needed.
Error Handling	Is error handling implemented effectively?	Yes	Error handling is implemented for API calls, but no user feedback is provided for errors.	Display a user-friendly error message or notification when fetching or updating data fails.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled effectively, but it could benefit from responsive design improvements.	Add responsive design to enhance usability on smaller screens.

Userorders.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name UserOrders follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., fetchOrders, setOrders) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

Housekeeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used in some parts of the code.	Extract inline styles into a separate CSS/SCSS file for better maintainability.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None.	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Potential redundant rendering checks for empty orders.	Optimize the conditional rendering logic for orders and loggedIn to avoid unnecessary renders.

Error Handling	Is error handling implemented effectively?	Yes	Error handling is implemented for the axios.get function.	Provide user-friendly feedback for errors, such as displaying a retry button.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	Styling is consistent, but responsiveness could be improved.	Add responsive design to ensure usability on smaller screens.

Women_Products.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name WomensProducts follows PascalCase.	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., fetchProducts, handleSearch) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

Housekeeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline styles are used extensively, which can make the code less maintainable.	Extract inline styles into a separate CSS/SCSS file for better maintainability and scalability.

Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	None	No changes needed.
Error Handling	Is error handling implemented effectively?	Yes	Error handling is implemented for the axios.get function, but no user feedback is provided for errors.	Display an error message or notification to users when products fail to load.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	The component is styled effectively, but it could benefit from responsive design improvements.	Add responsive design to enhance usability on smaller screens.

AuthConext.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	The component name AuthProvider follows PascalCase.	No changes needed.

2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names (e.g., handleLogin, setLoggedIn) follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

Housekeeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Inline state updates (e.g., setLoggedIn) could be grouped for better readability.	Refactor state updates into a single object or use a reducer for cleaner updates.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	None	No changes needed.
Error Handling	Is error handling implemented effectively?	No	There is no error handling for localStorage access, which could fail in certain browsers.	Add a try-catch block around localStorage operations for safer execution.
UI/UX Considerations	Are UI components styled effectively	Not Applicable	The component does not render UI elements directly.	No changes needed.

	and with consistency?			

index.js:

Java-Specific Code Review Checklist

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	Component names follow PascalCase (e.g., Manageorders, UserDashboard).	No changes needed.
2	Naming Conventions	Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Variable and method names follow camelCase.	No changes needed.
3	Naming Conventions	Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	Not Applicable	No constants are used in the provided code.	No changes needed.

House Keeping Checklist

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Repeated import paths (e.g., ../src/Components/...) can be replaced with aliasing for clarity.	Configure aliases in the Webpack or Vite setup for more readable and maintainable import paths.
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	None	No changes needed.

Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Some routes, such as /admin/support, are declared twice, which may cause unintended behavior.	Remove duplicate route declarations to avoid conflicts and ensure expected functionality.
Error Handling	Is error handling implemented effectively?	No	No error handling is present for undefined routes or invalid paths.	Implement a fallback route (e.g., 404 component) to handle undefined paths.
UI/UX Considerations	Are UI components styled effectively and with consistency?	Yes	Styling is centralized with index.css, but further modularization of styles could be beneficial.	Use CSS Modules or styled-components for component-specific styles.